REPUBLIC OF CAMEROON
Peace - Work - Fatherland
THE UNIVERSITY OF BAMENDA

P.O. Box 39 Bambili

NATIONAL HIGHER POLYTECHNIC INSTITUTE BAMENDA

(NAHPI)

REPUBLIQUE DU CAMEROUN
Paix - Travail - Patrie
UNIVERSITE DE BAMENDA

B.P. : 39 Bambili

ECOLE NATIONALE SUPERIEURE POLYTECHNIQUE DE BAMENDA
(ENSPB)

OBJECT ORIENTED DESIGN (OOD) AND OBJECT-ORIENTED PROGRAMMING (OOP)

SYSTEM ANALYSIS, DESIGN & IMPLEMENTATION (CAVE RUN GAME)

DEPARTMENT: COMPUTER ENGINEERING

COURSE: OBJECT ORIENTED DESIGN

OBJECT ORIENTED PROGRAMMING WITH C++

COURSE CODE: COME6105

COME6101

GROUP 5

TCHUIFO AKO ERASTUS	UBa25EP232
NDIFON TITIANA SIH	UBa23E3040

TABLE OF CONTENT

TABLE OF FIGURES

TABLE OF CODE SNIPPETS

CHAPTER 1 : INTRODUCTION

1.1 Background of the Study

Object-oriented analysis and design (OOAD) and object-oriented programming (OOP) are widely used to build modular, reusable, and maintainable software systems.
Game projects are commonly used in academia to teach OO concepts because they naturally involve interacting entities, state changes, and clear rules.

The “Cave Run” game specified in this assignment is a grid-based adventure where a player attempts to escape a cave while avoiding a monster and hazardous rooms.
The project requires not only working code but also complete OOAD documentation using UML diagrams and a corresponding C++ implementation, reflecting industry-style practice for analysis, design, and implementation alignment.

1.2 Problem Statement

Many beginner game implementations focus only on code and ignore proper requirements analysis, design documentation, and clear separation of concerns, which makes systems hard to understand, extend, and maintain.
In the context of this course, the challenge is to engineer a small but non-trivial game that systematically applies OOAD techniques and demonstrates good software engineering practices, not just “make it work” coding.

Specifically, the Cave Run project must:

- Capture and organize the game’s functional and non-functional requirements, including user interface behavior.
- Model the problem domain and solution using appropriate UML diagrams before coding.
- Implement the resulting design in C++ using inheritance, polymorphism, and robust error handling while providing a usable interface to the player.

1.3 Objectives of the Project

1.3.1 General Objective

The general objective of this project is to design and implement the Cave Run game using object-oriented analysis, object-oriented design, and object-oriented programming in C++, supported by a complete set of UML models and a functional user interface.

1.3.2 Specific Objectives

The specific objectives are:

- To identify and document the functional and non-functional requirements of the Cave Run game, including user interface requirements and constraints.
- To construct analysis and design models of the system using UML diagrams such as use case, activity (where needed), class, sequence, collaboration, and state diagrams.
- To design a coherent class structure that assigns clear responsibilities to Character, Player, Monster, Room and its subclasses, Map, and Game classes with low coupling and high cohesion.
- To implement the designed classes and interactions in C++, demonstrating inheritance, polymorphism, encapsulation, and good error-handling practices.
- To develop a simple, intuitive user interface (console-based or lightweight GUI) that allows the player to view the cave map, issue movement commands, and receive feedback on health, status, and game outcome.
- To test the system against key scenarios, verifying correct behavior of movement, hazardous rooms, monster logic, and end-game conditions.

1.4 Scope of the Project

The scope of this work is limited to a single-player, turn-based game played on a finite rectangular grid of rooms.
The map contains normal rooms, poisonous rooms, and trap rooms, with the player starting at the bottom-left and the exit at the top-right, while a monster moves on the same grid according to simple decision rules.

The project covers:

- Requirements specification, OO analysis, and OO design documented with UML diagrams.
- Implementation of the main entities and game rules in C++.
- A basic but usable user interface (console or lightweight GUI) and minimal error handling sufficient for smooth gameplay.

The project does not aim to provide:

- Networked or multiplayer gameplay.
- Advanced 2D/3D graphics, animations, or audio.

- Complex artificial intelligence beyond the described monster movement logic and basic randomization if needed.

1.5 Methodology and Tools Overview

The development follows an object-oriented software engineering process where analysis, design, and implementation are carried out in distinct but related phases.
Chapter 2 focuses on object-oriented analysis using textual requirements and analysis-level UML diagrams, Chapter 3 refines these into a design model with detailed UML diagrams, and Chapter 4 implements the design in C++.

In practical terms, the project uses:

- **Modelling tools:** A UML modelling tool (StarUML, PlantUML) to produce the required diagrams in standard notation.
- **Implementation tools:** A C++ compiler and IDE (e.g. g++/CMake, Visual Studio) to implement and test the game logic.
- **User interface technology:** Either console-based rendering with text and simple symbols or a lightweight GUI framework (such as SFML or Qt) depending on feasibility and course constraints.

1.6 Organization of the Report

The remainder of this report is organized into four chapters that mirror the OOAD–OOP workflow.

- **Chapter 2: Object-Oriented Analysis (OOA)** presents the system description, requirements, actors, use cases, and analysis-level models of the Cave Run game.
- **Chapter 3: Object-Oriented Design (OOD)** details the software architecture and design-level UML diagrams, including class, sequence, collaboration, and state diagrams for the key scenarios.
- **Chapter 4: Object-Oriented Programming (OOP)** describes the C++ implementation of the system, mapping the design model to code, including the game logic and user interface.
- **Chapter 5: Conclusion** summarizes the work, discusses limitations, and suggests possible extensions and future improvements to the Cave Run game.

CHAPTER 2 : OBJECT-ORIENTED ANALYSIS (OOA)

2.1 System Description and Environment

Cave Run is a turn-based grid game where a player attempts to escape from a cave composed of multiple rooms while avoiding a monster and hazardous rooms.
The cave is represented as a rectangular grid with a fixed start room at the bottom-left, an exit room at the top-right, and intermediate rooms that may be normal, poisonous, or trap rooms.

The player has a finite number of health points and can move up to two rooms per turn, while the monster moves one room per turn on the same grid.
The system is intended to run on a desktop computer using a C++ implementation with either a console-based or lightweight graphical user interface.

2.2 Stakeholders and Users

The main stakeholders and users of the system are:

- **Player (End User):** Interacts with the game, issues movement commands, and observes game status.
- **Instructor / Evaluator:** Assesses correctness of requirements, analysis, design, and implementation, and the quality of the user interface.
- **Developers (Student Group):** Design, implement, and maintain the game according to the assignment specification.

These stakeholders expect the system to behave according to the specified rules, provide clear feedback, and be structured in a way that can be understood and extended.

2.3 Functional Requirements

The core functional requirements of Cave Run are summarized below; numbering can be reused in later chapters.

- **Start Game:**
  The system shall allow the player to start a new game, which initializes the map, player, and monster positions.
- **Display Game State:**
  The system shall display the cave grid, the positions of the player and monster, and key information such as health points and remaining moves.
- **Player Movement:**
  The system shall allow the player, on their turn, to move up to two adjacent rooms (including diagonals), subject to map boundaries.

- **Monster Movement:**
  The system shall move the monster one room per turn based on its decision logic using the player and exit positions.
- **Room Effects:**
  The system shall apply room-specific effects when the player enters a room (no effect for normal rooms, poison effects for poisonous rooms, immediate damage and end of turn for trap rooms).
- **Health and Status Update:**
  The system shall update the player's health and status (e.g. poisoned) after each relevant action or time step and display the updated status.
- **Game Over Detection:**
  The system shall detect game over when the player reaches the exit room, when the monster enters the same room as the player, or when the player's health is fully depleted.
- **Error Handling for User Input:**
  The system shall handle invalid user inputs (e.g. impossible moves, invalid commands) by rejecting them and prompting the user without crashing.
- **Exit Application:**
  The system shall allow the player to terminate the game gracefully and return to the operating system.

2.4 Non-Functional and UI Requirements

2.4.1 Non-Functional Requirements

- **Usability:**
  The interface shall clearly indicate the current turn, player health, remaining moves, and game messages so that a new user can understand the game without lengthy instructions.
- **Reliability and Robustness:**
  The system shall avoid abnormal termination and handle invalid inputs, out-of-bounds movements, and inconsistent states gracefully.
- **Performance:**
  For a typical map size used in this project, the system shall respond to a user command within a fraction of a second on standard hardware.
- **Maintainability and Extensibility:**
  The design shall allow adding new room types or modifying monster behavior with minimal changes to existing classes by relying on polymorphism and clear interfaces.

2.4.2 User Interface Requirements

- **Map Display:**
  The system shall show the cave as a grid, using distinct symbols or colors for the player, monster, start room, exit room, and hazardous rooms.
- **Status Display:**
  The system shall display the player's current health points, poisoned status, and remaining moves each turn in a dedicated area of the interface.
- **Input Mechanism:**
  The system shall allow the player to specify movement commands using either keyboard input (e.g. directional commands) or mouse clicks, depending on the chosen UI technology.
- **Feedback and Messages:**
  The system shall provide clear text messages when the player enters a hazardous room, takes damage, is poisoned, is cured, or when the game is won or lost.

2.5 Actors and Use Case Model

2.5.1 Identified Actors

- **Player:** Human user who starts the game, issues movement commands, and receives feedback.
- **Game System:** Automated system that maintains game state, applies rules, and interacts with the player through the interface.

2.5.2 Main Use Cases

The principal use cases of the Cave Run system are:

- **Start New Game**
  The player starts a new game; the system initializes the map, positions, and initial status.
- **Play Turn (Player Move)**
  The player chooses movement up to two rooms; the system validates and executes the moves and applies any room effects.
- **Monster Turn**
  After the player's turn, the system moves the monster according to its decision rules.
- **View Game Status**
  At any time during gameplay, the player views the map, positions, and status information.

- **End Game**
  The system determines that the player has won or lost and displays the final result.
- **Exit Application**
  The player exits the application from the interface.
- **Visit Room**
- **Get poisoned**

2.5.3 Use Case Diagram

The relationships between the Player, Game System, and the use cases are summarized in the use case diagram.

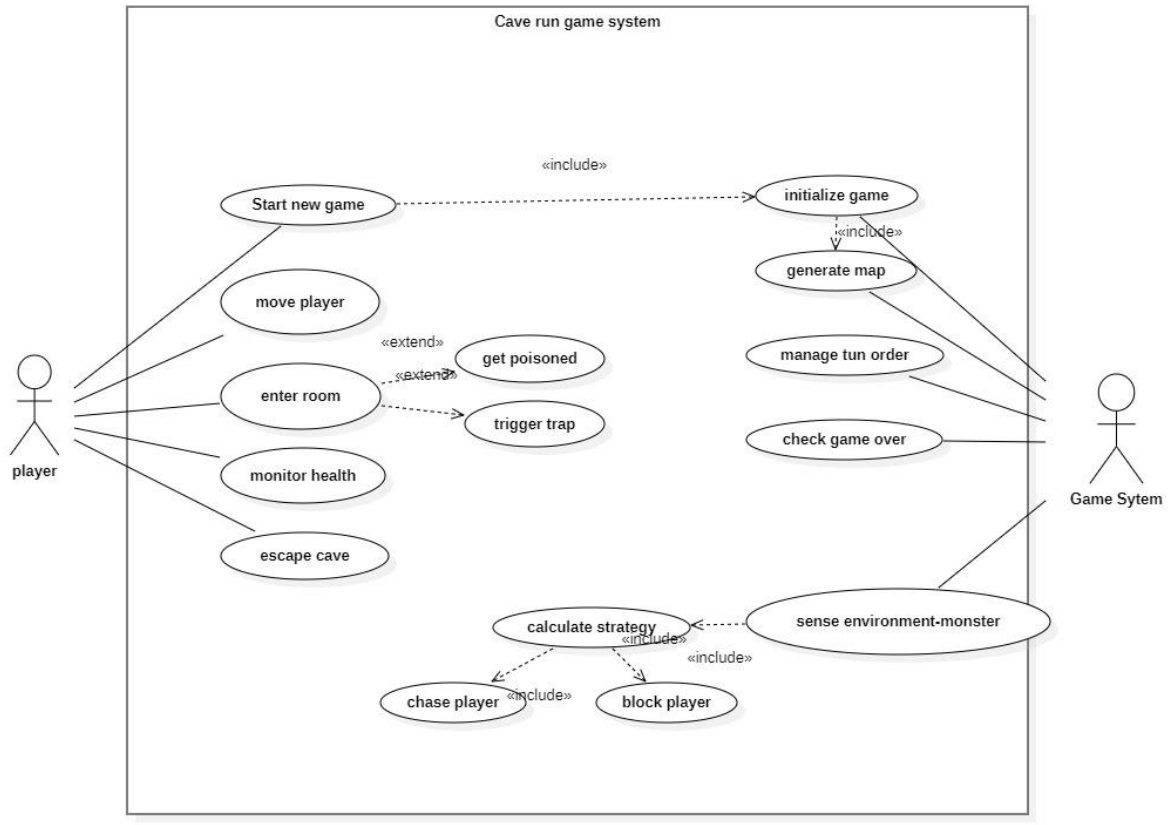


Figure 2.1 Use Case Diagram for Cave Run

2.6 Use Case Descriptions

This section gives brief structured descriptions for the most important use cases. A tabular format can be used in your final document.

2.6.1 Start New Game

- **Actor:** Player
- **Preconditions:** Application is running; no game or a previous game has ended.
- **Main Flow:**
    1. Player selects “Start New Game”.
    2. System initializes the map, start room, exit room, hazardous rooms, player, and monster positions.
    3. System sets player health and remaining moves to initial values and clears any status flags.
    4. System displays the initial game state.
- **Postconditions:** A new game session is ready and the player’s first turn can begin.

2.6.2 Play Turn (Player Move)

- **Actor:** Player
- **Preconditions:** A game is running; it is the player’s turn; player is alive.
- **Main Flow:**
    1. System displays current map and status.
    2. Player issues movement commands (up to two steps) through the UI.
    3. System validates each requested move (adjacency, boundaries).
    4. System moves the player and applies any room effects (poison, traps, etc.).
    5. System updates player health, poisoned status, and remaining moves.
    6. When remaining moves reach zero or the player ends the turn, control passes to the monster turn.
- **Postconditions:** Player position and status are updated; game may proceed to monster turn or end if a game-over condition is reached.

2.6.3 Monster Turn

- **Actor:** Game System
- **Preconditions:** A game is running; the player’s turn has ended and the player is still alive.
- **Main Flow:**

1. System determines the monster's movement direction based on player and exit positions.
2. System moves the monster one room if the move is valid.
3. System checks if the monster is now in the same room as the player.

- **Postconditions:** Monster position is updated; the system either triggers game over (if the player is caught) or returns control to the player for the next turn.

2.6.4 End Game (Win or Lose)

- **Actor:** Game System, Player
- **Preconditions:** A game-over condition is met (player reaches exit, player health is zero, or monster catches player).
- **Main Flow:**
  1. System detects the game-over condition.
  2. System displays a message indicating whether the player has won or lost, along with key statistics (e.g. remaining health or number of moves).
  3. Player can choose to start a new game or exit the application.
- **Postconditions:** Game session is completed; system is either ready for a new game or to exit.

2.7 Analysis Model (Conceptual Classes)

The analysis model identifies the main conceptual classes and their high-level relationships without going into C++ implementation details.

Key conceptual classes include:

- **Game:** Manages the overall game session, including turn control and game-over checks.
- **Map:** Represents the grid of rooms and provides access to rooms by position.
- **Room (and Subclasses):** Abstract concept of a location that can be entered by characters; specialized by NormalRoom, PoisonRoom, and TrapRoom.
- **Character (and Subclasses):** Abstract entity that occupies rooms and moves; specialized by Player and Monster.
- **Position:** Represents coordinates of rooms and characters within the map grid.
- **Health / Status:** Conceptual representation of the player's health points and states such as poisoned.

2.7.1 Conceptual Class Diagram

The relationships between these conceptual classes can be represented in a high-level class diagram that focuses on associations and inheritance.

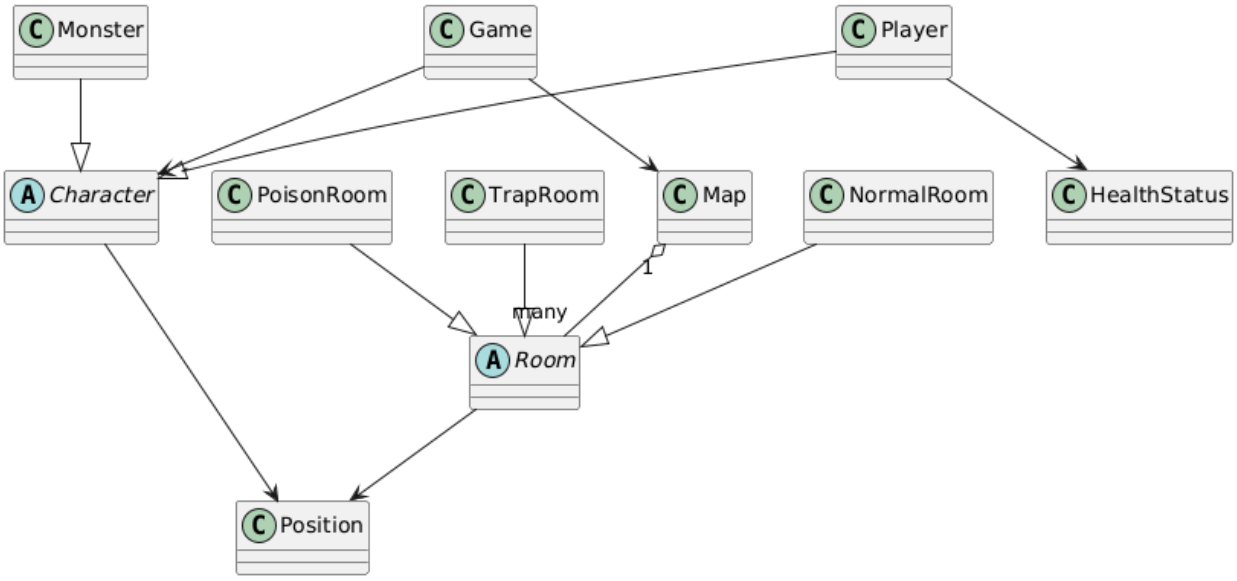


Figure 2.2: Conceptual Class Diagram for Cave Run

CHAPTER 3 : OBJECT-ORIENTED DESIGN (OOD)

3.1 Design Approach and Principles

The object-oriented design of Cave Run refines the analysis model into concrete software structures that can be implemented in C++.
The design follows key principles such as encapsulation, low coupling, high cohesion, and clear separation between game logic and the user interface.

Responsibilities are assigned primarily to domain classes (Game, Map, Character, Room, etc.) rather than to procedural controllers, which supports extensibility and easier testing.
Polymorphism is used so that different room types and characters can specialize behavior without changing the game loop logic.

3.2 Overall Architecture

The system is structured into three main layers:

- **Game Logic Layer:** Contains the core domain classes: Game, Map, Character and its subclasses, Room and its subclasses, Position, and health/status tracking.
- **User Interface Layer:** Handles input and output through either a console or a lightweight GUI, rendering the map and messages based on the state managed by the game logic.
- **Utility Layer (optional):** Provides supportive functionality such as randomization, configuration parameters, or small helper classes where needed.

The Game class acts as the main coordinator, using the Map and Character objects to implement the rules defined in Chapter 2 while remaining independent of specific UI technology.
UI components query Game for state and send user commands back to it, preserving a clear separation between the internal model and presentation.

3.3 Static Design: Class Identification and Responsibilities

3.3.1 Main Classes and Responsibilities

From the requirements and use cases, the key design classes and their responsibilities are as follows.

- **Game**
    - Orchestrates a complete game session (initialization, turn loop, and termination).
    - Creates and owns the Map, Player, and Monster objects.

- Alternates turns between Player and Monster, invokes character actions (sense, move, update), and checks for win or loss conditions.

- **Map**
  - Represents the 2D grid of rooms forming the cave.
  - Stores references to Room objects in a 2D structure and provides lookup operations by grid coordinates.
  - Knows the grid dimensions and the positions of the start and exit rooms to support boundary checks and win-condition evaluation.
- **Room (abstract)**
  - Base type for all room variants.
  - Stores its grid position and display attributes such as color or label.
  - Declares a virtual visit(Player) operation that is overridden by subclasses to implement specific effects when the player enters the room.
- **NormalRoom (subclass of Room)**
  - Represents a safe room with no special effect.
  - Overrides visit(Player) without changing health or status, leaving the player unchanged except for position.
- **PoisonRoom (subclass of Room)**
  - Represents a poisonous room that poisons the player on visit.
  - Overrides visit(Player) to mark the player as poisoned and possibly configure ongoing damage per update; the room's display is changed (e.g. to green) after being visited to indicate hazard.
- **TrapRoom (subclass of Room)**
  - Represents a trap that inflicts a fixed amount of damage and immediately ends the player's turn.
  - Overrides visit(Player) to subtract health and set the player's remaining moves to zero; the room's display is changed (e.g. to red) after being visited.
- **Character (abstract)**
  - Base type for movable entities (Player and Monster).
  - Stores current position and the number of remaining moves in the current turn.

    - Provides generic movement logic and declares abstract or overridable operations for sensing game state, deciding movement direction, and performing per-turn updates.

- **Player (subclass of Character)**
    - Adds attributes for health points, poison status, poison damage per update, and probability of curing poison.
    - Moves based on user input and tracks the rooms visited; can make up to two moves per turn, with this allowance reset at the beginning of each player turn.
    - In update(), applies ongoing poison damage if poisoned and attempts to cure poison according to the configured probability.
- **Monster (subclass of Character)**
    - Moves autonomously by sensing both the player's position and the exit room position.
    - On its turn, compares its distance to the exit with the player's distance to the exit; if the monster is closer to the exit, it moves towards the player, otherwise it moves towards the exit, advancing one step per turn.
- **Position**
    - Represents an (x, y) coordinate on the map grid using integer components.
    - Used by both Room and Character to identify locations and compute adjacency and distances.
- **HealthStatus**
    - Represents the player's health points and high-level status flags such as "poisoned" or "dead".
    - Provides simple operations to reduce health, check for zero or below, and update status.

Each class has a focused responsibility, which helps keep the design modular and easier to extend, for example by adding new room types or alternative character behaviors.

3.3.2 Design-Level Class Diagram

The design-level UML class diagram includes the main classes, their attributes and operations (at least at a high level), and relationships such as inheritance and associations.

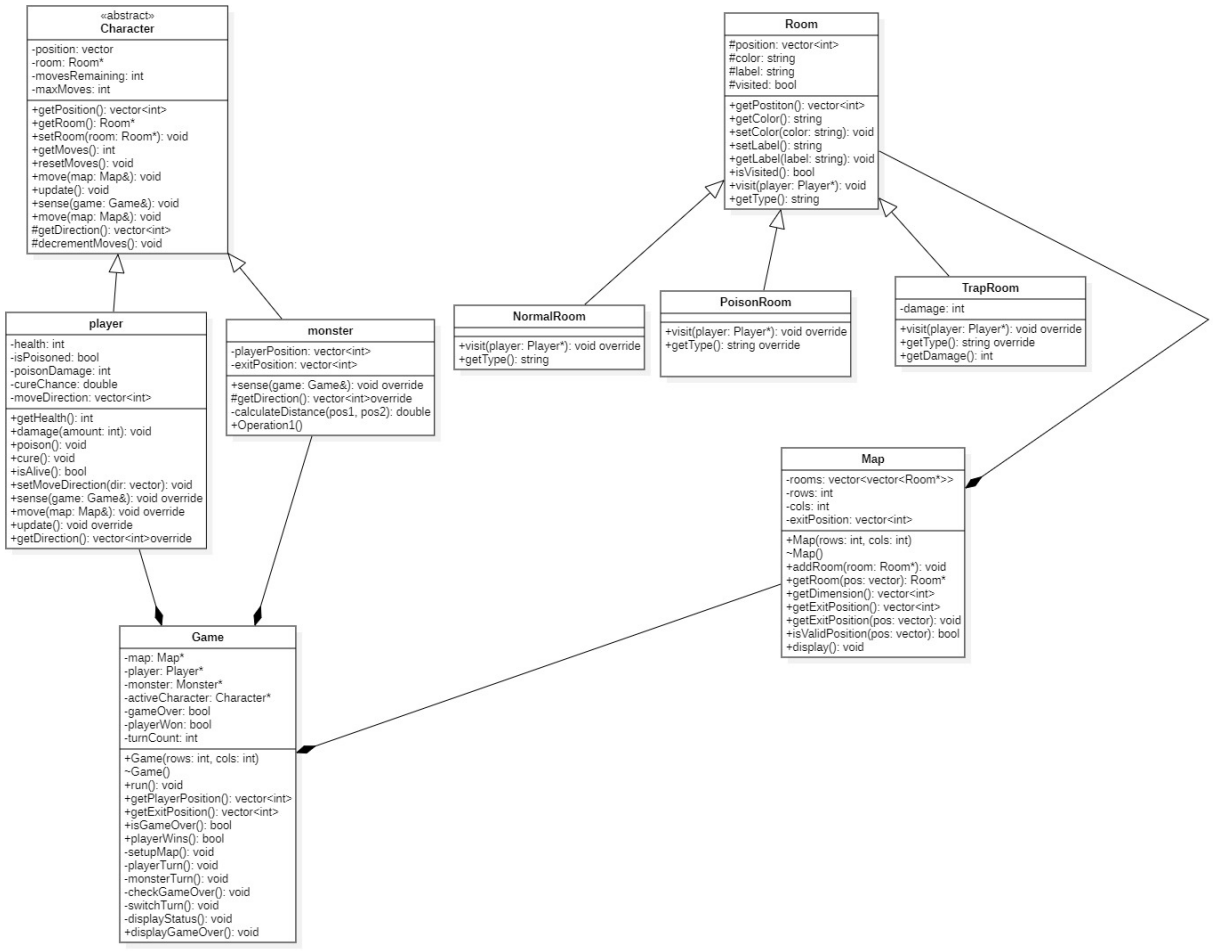


Figure 3.1 Design-Level Class Diagram for Cave Run

3.4 Dynamic Design: Interaction Diagrams

To model the interactions between objects during key scenarios, sequence and collaboration diagrams are used.

3.4.1 Sequence Diagram – Player move

This sequence diagram describes the main flow of Play Turn , including movement and room effects.

- Player (actor) issues a movement command to the Game through the UI.
- Game asks Map for the target Room based on the requested Position.
- Game validates the move (bounds, adjacency) and, if valid, instructs Player to move to the new Position and decrement remaining moves.

- Game invokes visit(Player) on the target Room, which is dynamically dispatched to NormalRoom, PoisonRoom, or TrapRoom as appropriate.
- Room updates the Player's health and status and, if necessary, ends the player's turn (in the case of TrapRoom).

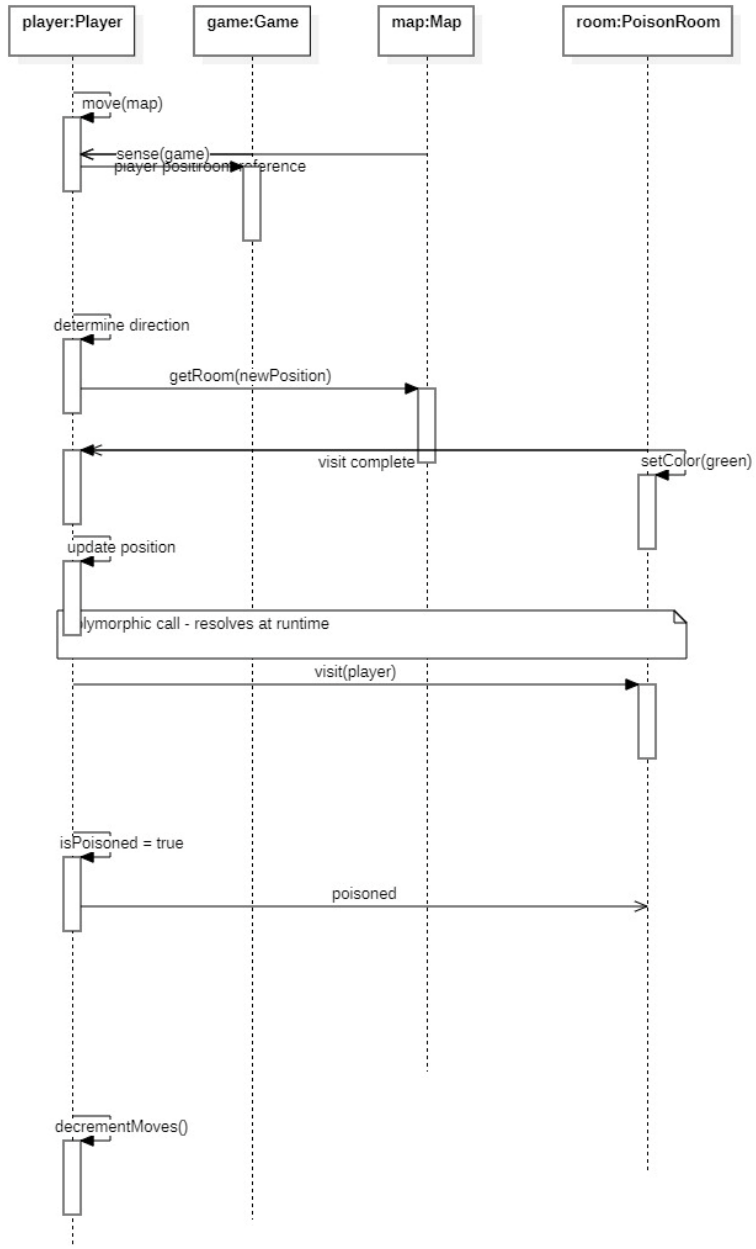


Figure 3.2 Sequence Diagram – Player Turn

This diagram shows how polymorphism is used: Game does not need to know which type of Room is entered; it simply calls visit(Player).

3.4.2 Sequence Diagram – Monster Turn

This sequence diagram captures "Monster Turn".

- Game invokes sense(Game) on the Monster, allowing it to obtain the Player and exit positions.
- Monster calculates its preferred movement direction based on distance to player and exit.
- Game requests Map to verify the target Position and then calls Monster's move operation to update its Position.
- Game checks whether Monster and Player occupy the same Room and triggers a game-over condition if they do.

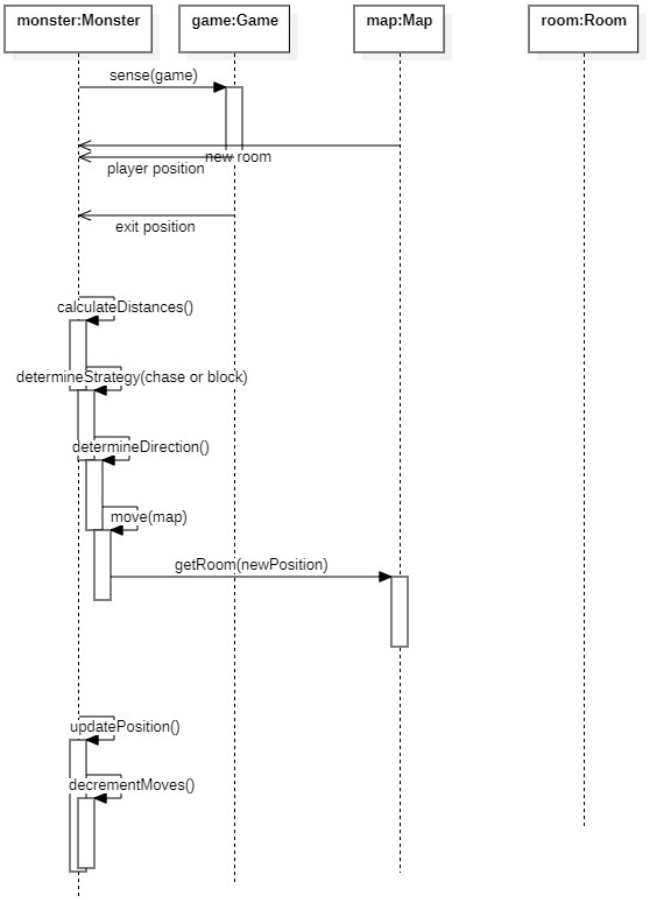


Figure 3.3 Sequence Diagram – Monster Turn

3.4.3 Sequence Diagram – Game Initialization

To complete the key flows,

- Player selects “Start New Game”.
- Game constructs Map, creates Rooms (including hazardous ones), and creates Player and Monster objects.
- Game enters the main loop, alternating turns until a win or loss condition is detected, then signals the display of a final result.

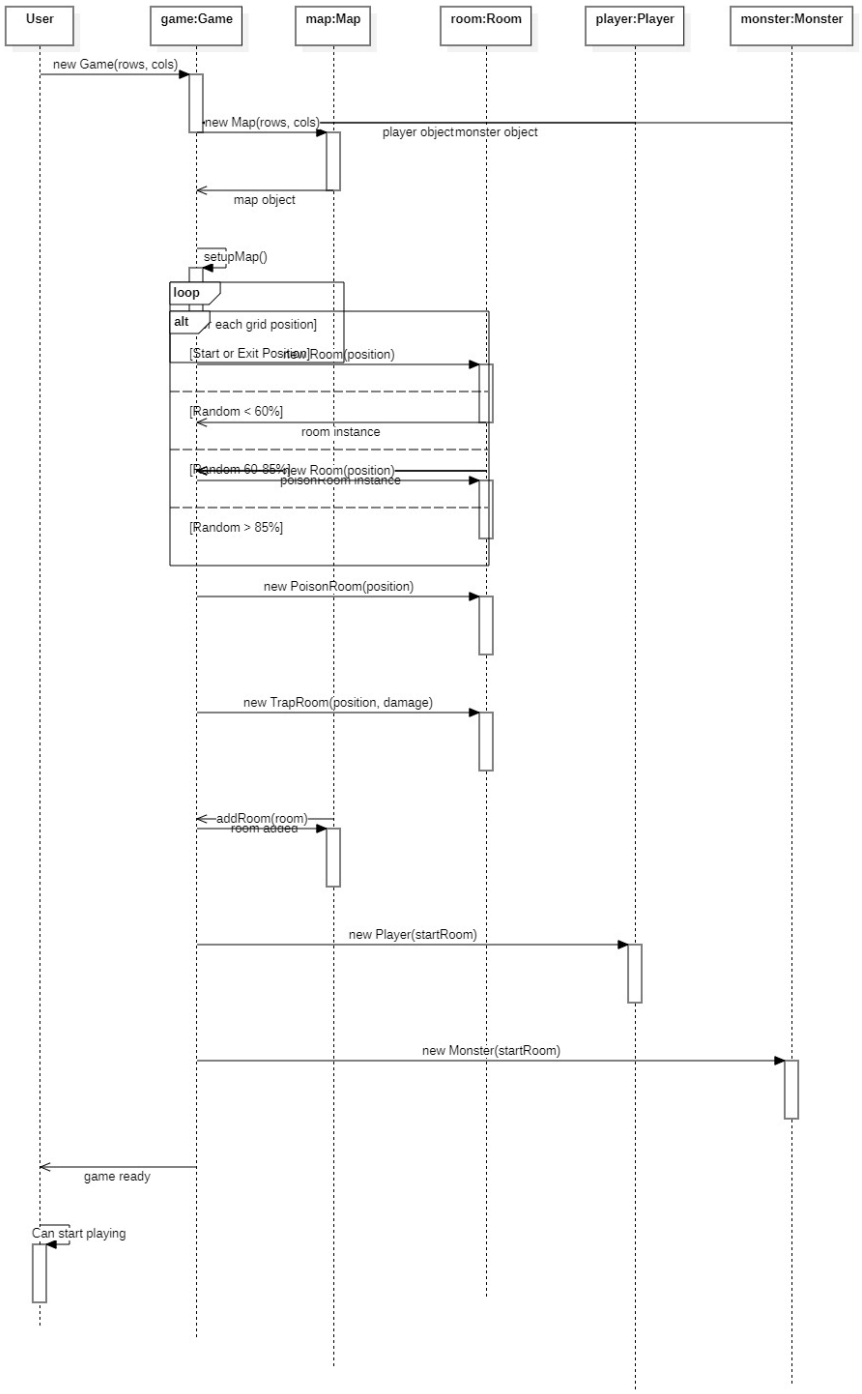
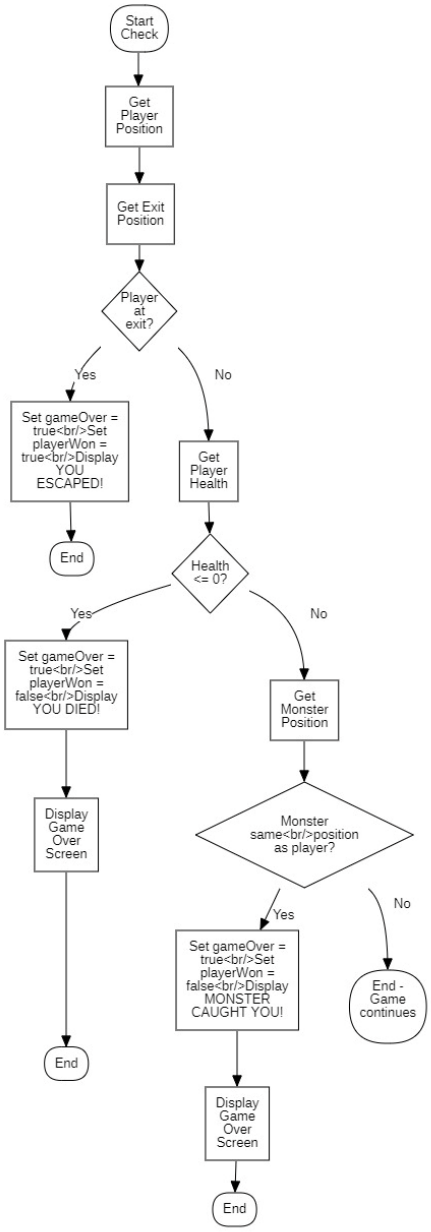


Figure 3.4 Sequence Diagram – Game Initialization

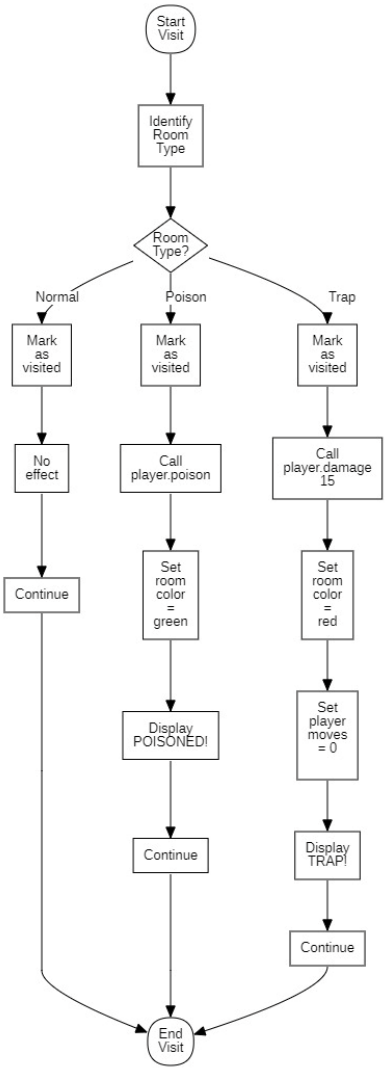
3.4.3 Sequence Diagram – get poisoned

Activity Diagrams



Check game over

Visit a room



3.4.4 Collaboration (Communication) Diagrams

Communication diagrams for the same scenarios highlight links between objects and the order of messages rather than time on a vertical axis.

- **Communication Diagram – Player Turn**
    - Shows Game, Map, Player, and Room instances, with numbered messages indicating movement and visit(Player) calls.

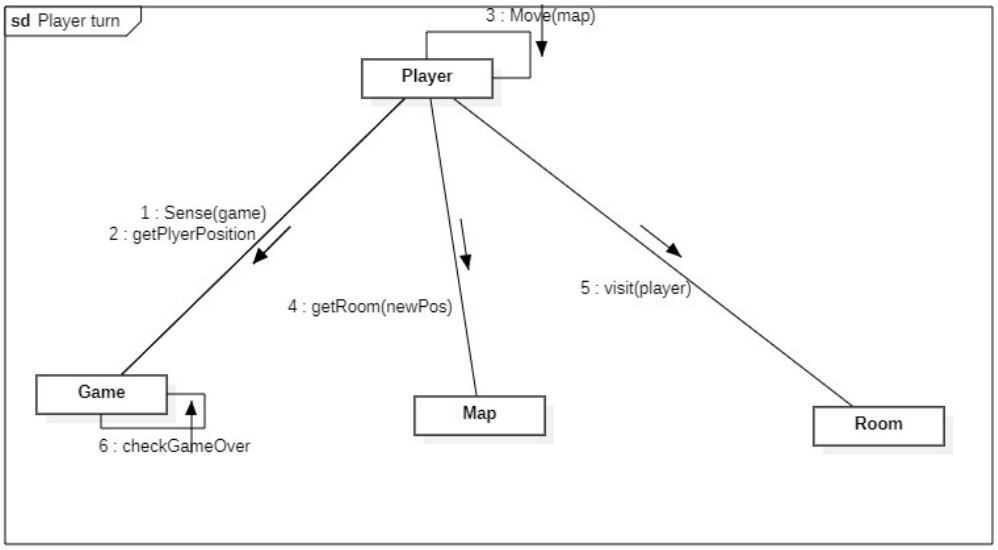


Figure 3.5 Communication Diagram – Player Turn

- **Communication Diagram – Update Game State**

  Shows object collaboration when updating game state. Objects involved are game:Game, activeChar:Character*, player:Player, monster:Monster and polymorphic update and turn switching.

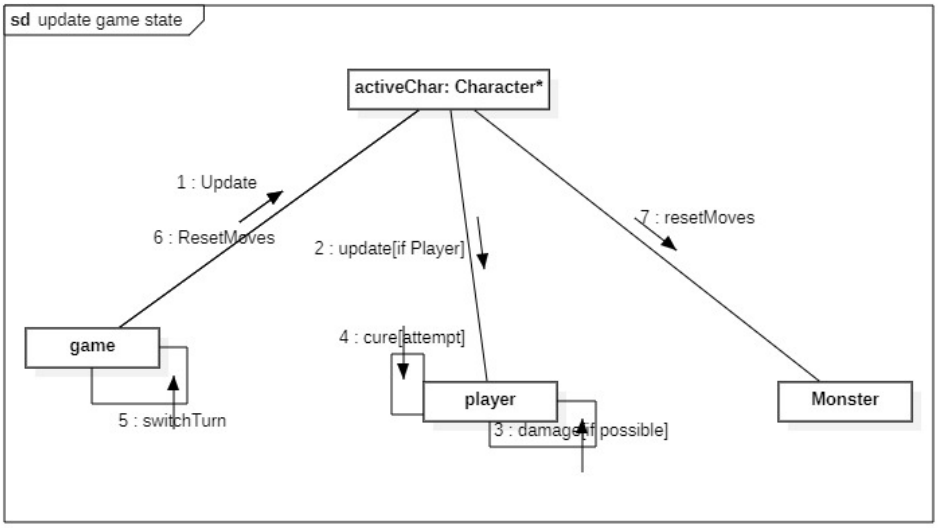


Figure 3.6 Communication Diagram – Update game state

- **Communication Diagram – Monster Turn**

Shows Game, Map, Monster, and Player instances, with messages for sensing positions and moving the Monster.

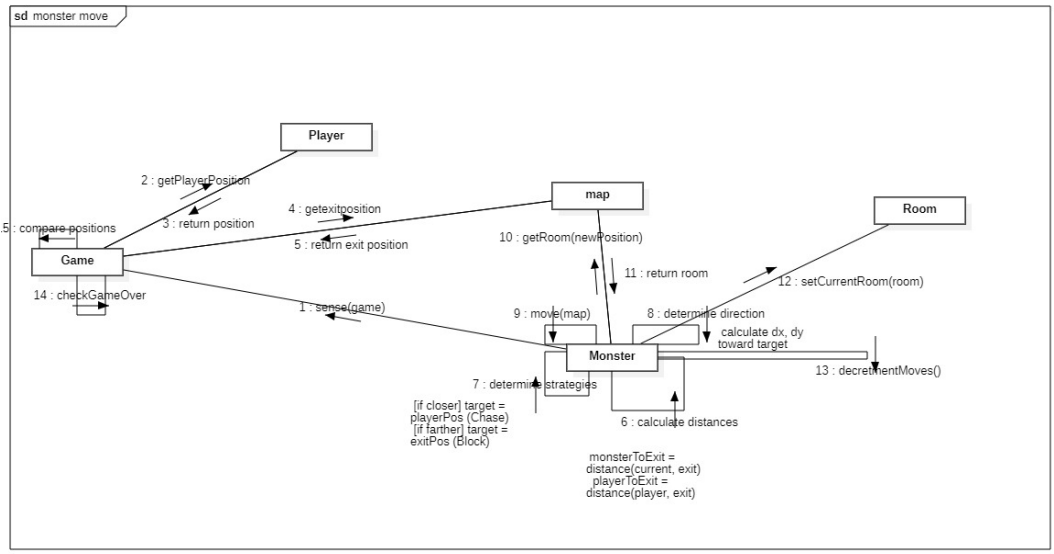
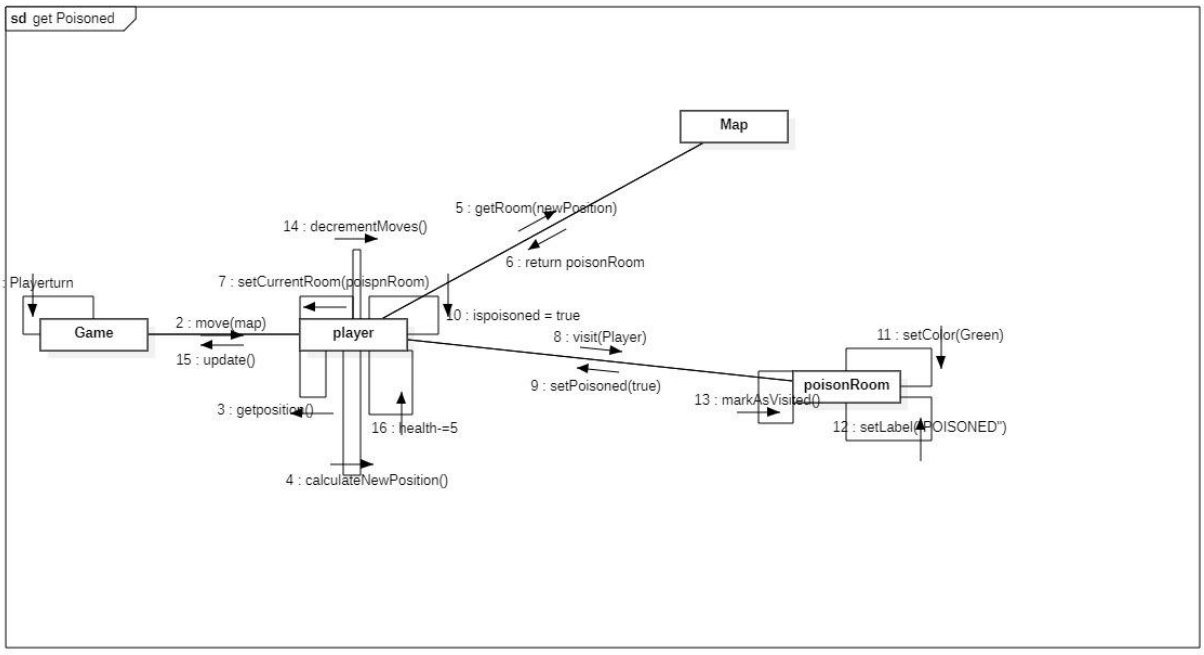


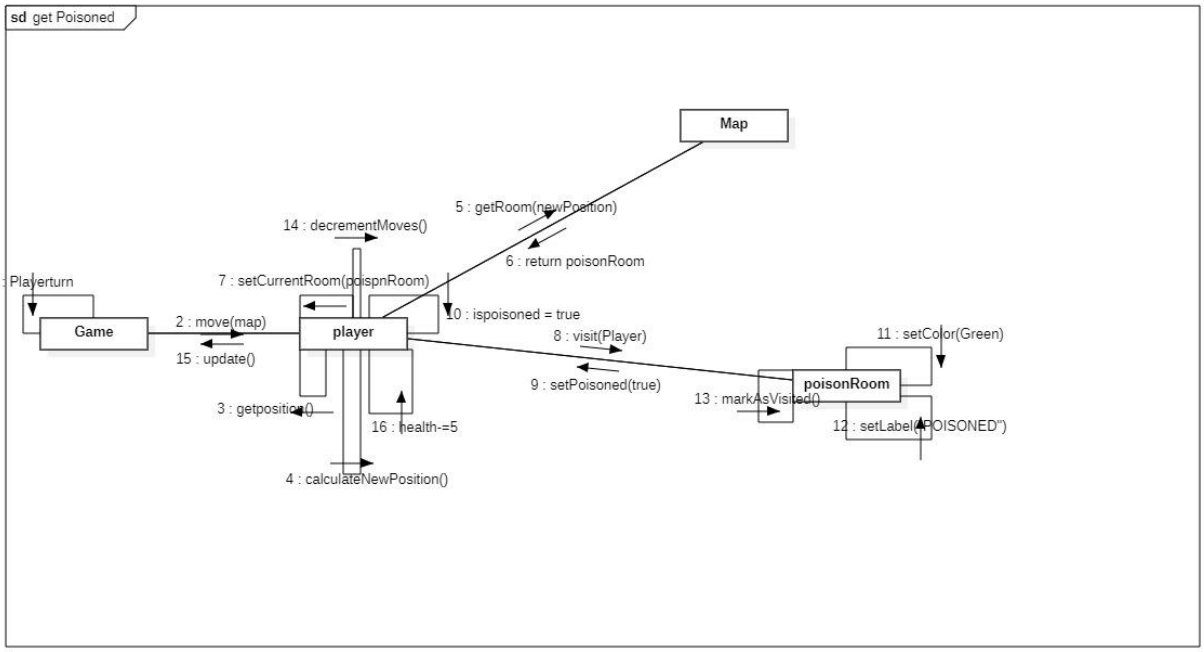
Figure 3.6 Communication Diagram – Monster Turn

These diagrams complement the sequence diagrams by emphasizing structural relationships during interactions.

- **Communication Diagram – Monster Turn**



- **Communication Diagram –get poisoned**



3.5 State-Based Design: State Diagrams

State diagrams are used to model how objects change their internal state over time in response to events.

3.5.1 Player State Diagram

The Player has distinct logical states driven mainly by health and poison effects.

- **State Diagram – Player**
    - **Healthy:** Normal status, takes damage only from traps or other effects.
    - **Poisoned:** Entered when visiting a PoisonRoom; on each update, health decreases by a configured amount and there is a chance to return to Healthy if the poison is cured.
    - **Dead:** Entered when health is reduced to zero or below; this triggers game over if the player is the active character.

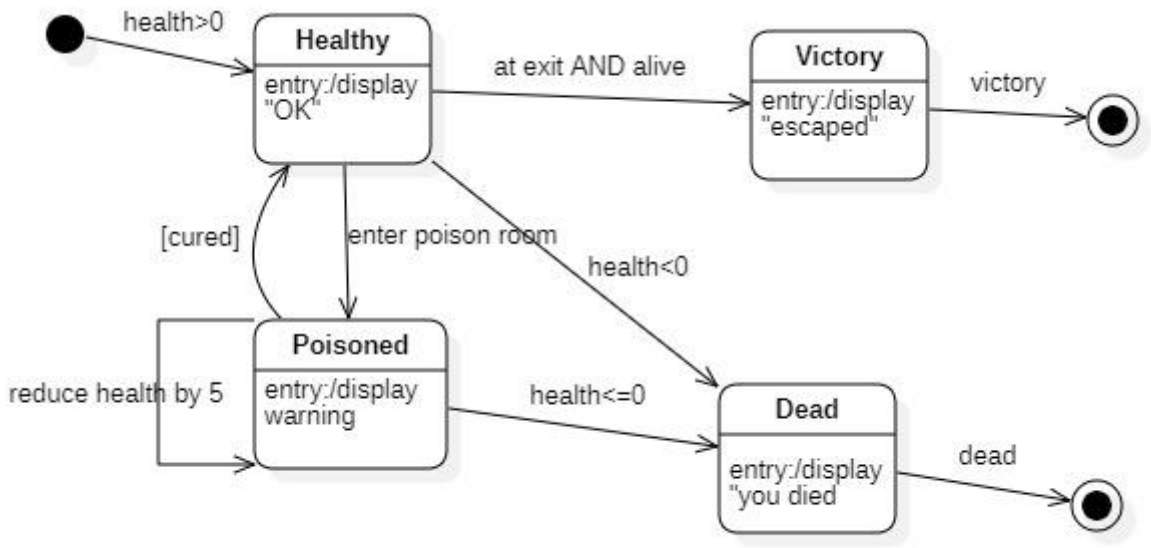


Figure 3.7 State Diagram – Player

Transitions are triggered by events such as “visit PoisonRoom”, “update with poison damage”, and “health ≤ 0”.

3.5.2 Game State Diagram

The Game object progresses through high-level control states during a session.

- **State Diagram – Game**

Shows game state transitions during execution.
States: Initializing, PlayerTurn, MonsterTurn, GameOver.

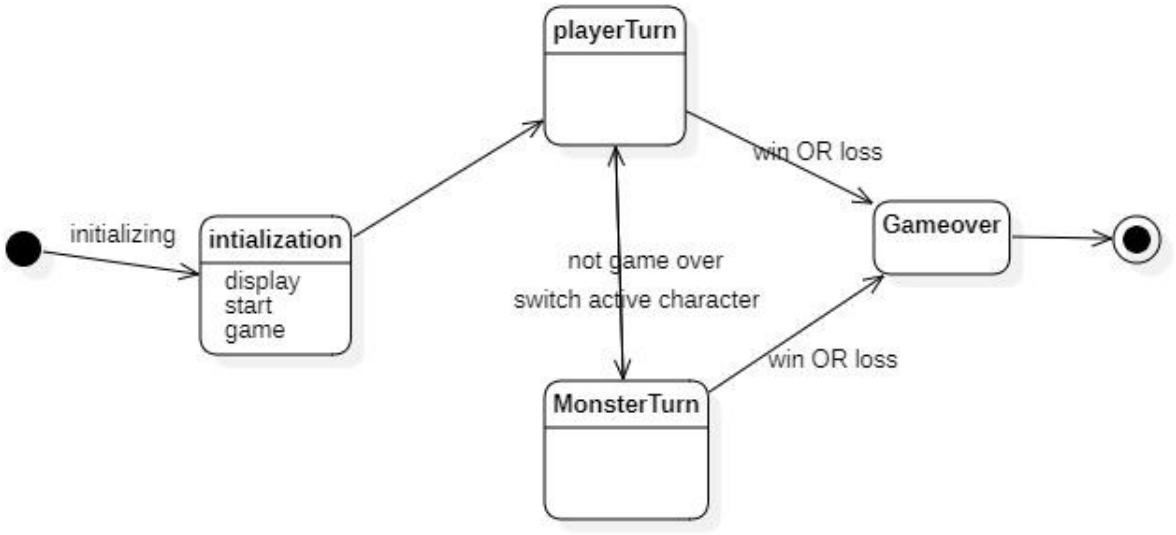


Figure 3.8 State Diagram – Game

Events such as “initialization complete”, “win detected”, and “loss detected” cause transitions between these states.

3.6 User Interface Design

The user interface design focuses on presenting the grid, status information, and messages in a simple and readable form.

- **Layout:** A main area shows the cave grid, with symbols or colors representing the player, monster, start, exit, and hazardous rooms; a secondary area displays health points, poisoned status, remaining moves, and textual messages.
- **Interaction:** During the player’s turn, the interface accepts movement commands (keyboard or mouse) and forwards them to the Game object; after each move, the display is refreshed to reflect new positions and statuses.

The UI design intentionally separates presentation from game logic so that the underlying classes can be reused with either a console-based view or a graphical window without changing core behavior.

3.7 Error Handling and Robustness Design

Error handling is integrated into the design to prevent invalid user input or inconsistent states from crashing the system.

- **Input Validation:** The Game validates all movement commands for adjacency and map boundaries before updating positions; invalid moves result in user feedback and no state change.

- **Defensive Checks:** Map ensures that any requested room coordinate is within valid bounds before returning a Room reference, and Character methods verify that remaining moves are positive before performing actions.

This design supports graceful recovery from common user errors and simplifies debugging by centralizing critical checks in a small number of classes.

CHAPTER 4 : OBJECT-ORIENTED PROGRAMMING (OOP IMPLEMENTATION)

4.1 Implementation Environment and Tools

The Cave Run game is implemented in C++ using classes, inheritance, virtual functions, and dynamic binding to realize the object-oriented design defined in Chapter 3.
Compilation and testing can be performed with a standard C++ toolchain, such as g++ with a simple build script or an IDE like Code::Blocks or Visual Studio.

The same core design supports either a console-based interface or a lightweight graphical interface, depending on the libraries available and course constraints.
In the console variant, the grid and status information are rendered with text and optional ANSI colors, while a graphical variant can rely on a 2D library such as SFML for window, drawing, and event handling.

4.2 Mapping Design Classes to C++ Code

Each UML class from Figure 3.1 is implemented as a C++ class, usually with separate header and source files for clarity and modularity.
The key source files include Game, Map, the Character hierarchy (Character, Player, Monster), the Room hierarchy (Room, NormalRoom, PoisonRoom, TrapRoom), and basic helper classes like Position.

A typical organization is:

- Game.h / Game.cpp: game control, initialization, main loop, and game-over checks.
- Map.h / Map.cpp: room grid, coordinate handling, and start/exit positions.
- Character.h, Player.h, Monster.h (+ corresponding .cpp files): movement logic and per-turn updates.
- Room.h, NormalRoom.h, PoisonRoom.h, TrapRoom.h (+ .cpp): polymorphic room effects invoked from the game loop.

This structure closely follows the UML class diagram, making it easy to navigate from documentation to code and back.

4.3 Implementation of Core Classes

4.3.1 Character, Player, and Monster

The abstract base class Character encapsulates common data and behavior for movable entities, while Player and Monster provide specialized implementations.

Snippet 4.1: Character base class and Player inheritance

```
// Character.h
class Game;
class Map;
class Position;

class Character {
protected:
    Position position;
    int remainingMoves;

public:
    Character(const Position& startPos, int movesPerTurn);
    virtual ~Character() = default;

    const Position& getPosition() const;
    void setPosition(const Position& pos);

    void resetMoves();
    bool hasMovesLeft() const;

    virtual void sense(const Game& game) = 0;
    virtual Position decideMove(const Game& game) = 0;
    virtual void update() {}

    void move(const Position& target, const Map& map);
};
```

```
//Player.h
class Player : public Character {
private:
    int health;
    bool poisoned;
    int poisonDamage;
    double cureProbability;

public:
    Player(const Position& startPos);

    void damage(int amount);
    bool isDead() const;

    void setPoisoned(bool flag);
    bool isPoisoned() const;

    void sense(const Game& game) override;
    Position decideMove(const Game& game) override;
    void update() override;           // applies poison damage and cure
chance
};
```

The Player inherits from Character and overrides sense, decideMove, and update to implement user-controlled movement and poison effects, while reusing the base movement logic.
The Monster class similarly inherits from Character and overrides decision logic to implement the distance-based chase behavior described in Chapter 3.

4.3.2 Room Hierarchy and Room Effects

The Room hierarchy implements different room behaviors using polymorphism through a virtual visit(Player&) function.

Snippet 4.2 Room base class and specialized room effects

```
//room.h
class Room {
protected:
    Position position;
    bool visited;

public:
    Room(const Position& pos);
    virtual ~Room() = default;

    const Position& getPosition() const;
    bool isVisited() const;
    void markVisited();

    virtual void visit(Player& player) = 0;  // pure virtual
};
```

```
// PoisonRoom.h
class PoisonRoom : public Room {
private:
    int poisonDamage;

public:
    PoisonRoom(const Position& pos, int dmg);

    void visit(Player& player) override;     // poisons the player
};
```

```
// PoisonRoom.cpp
void PoisonRoom::visit(Player& player) {
    player.setPoisoned(true);
    // configure ongoing damage if needed
    markVisited();
}
```

```
// TrapRoom.h
class TrapRoom : public Room {
private:
```

```
    int damageAmount;

public:
    TrapRoom(const Position& pos, int dmg);

    void visit(Player& player) override;     // damage and end turn
};
```

```
// TrapRoom.cpp
void TrapRoom::visit(Player& player) {
    player.damage(damageAmount);
    player.resetMoves();    // or set remainingMoves to 0 to end turn
    markVisited();
}
```

At runtime, the Game class holds pointers or references to Room but calls visit polymorphically, which ensures the correct effect is applied without type checks.

4.3.3 Game and Map

The Game class implements the main loop and delegates map- and room-related tasks to Map and the room hierarchy.

Snippet 4.3 Using polymorphism in the game loop

```
// Game.cpp
void Game::processPlayerTurn() {
    while (!gameOver && player.hasMovesLeft()) {
        // get movement command from UI
        Position target = ui->getPlayerMove();

        if (!map.isInside(target)
         || !map.isAdjacent(player.getPosition(), target)) {
            ui->showError("Invalid move");
            continue;
        }

        Room* room = map.getRoom(target);
        player.move(target, map);

        room->visit(player); // polymorphic call

        player.update();     // apply poison and other effects
        checkGameOver();

        ui->refresh(map, player, monster);
        if (!player.hasMovesLeft() || gameOver) break;
    }
}
```

The Map class manages room storage and coordinate checks, typically using a 2D container such as std::vector<std::vector<Room*>>, and provides helper methods like isInside, getRoom, and accessors for start and exit positions.

4.4 User Interface Implementation

The user interface layer interacts with the Game class through a small, well-defined set of methods, without directly manipulating internal game data structures.
For console-based implementations, a typical pattern is:

- Read user input (e.g. “W/A/S/D”, or coordinates).
- Translate input into a Position or direction and pass it to Game methods such as processPlayerTurn.
- After each action, request the current map and status from Game and redraw the grid and text-based status.

In a graphical implementation, the UI would run an event loop that captures keyboard or mouse events, calls Game operations, and uses drawing functions to render the updated map and entities in a window.

4.5 Use of Inheritance, Polymorphism, and Encapsulation

Cave Run’s implementation demonstrates object-oriented principles directly tied to the UML design.

- **Inheritance:**
    - Player and Monster inherit from Character, sharing movement and state while customizing sensing and decision logic.
    - NormalRoom, PoisonRoom, and TrapRoom inherit from Room, sharing position and visited state while customizing visit(Player&).
- **Polymorphism:**
    - The visit function is called through Room* in the game loop, so the effect depends on whether the concrete room is normal, poisonous, or a trap.
    - The update and decision methods of Character subclasses are invoked via base-class pointers, enabling different behaviors for Player and Monster without changing the main loop.

- **Encapsulation:**
    - Internal details such as health, poison flags, and room hazards are kept private to their classes and exposed only through controlled public methods, preserving invariants and simplifying reasoning.

These features make it straightforward to extend the game, for example by adding a HealingRoom class that restores health, without modifying existing Game loop logic.

4.6 Error Handling and Robustness in Code

Input validation and boundary checks are implemented in Game and Map before any state changes are made.
Invalid moves, such as leaving the map or attempting to move more than the allowed distance, are rejected and reported to the player through UI messages rather than causing crashes.

Map operations such as getRoom are guarded by isInside checks, and any unexpected conditions can be detected during development using simple assertions.
This defensive style supports the reliability and robustness requirements defined in Chapter 2 and helps maintain consistency between the implementation and the design models.

4.7 Testing and Validation

Testing focuses on verifying that the implementation behaves consistently with the use cases, sequence diagrams, and state diagrams defined earlier.

- **Unit-level checks:**
    - Movement validation methods (isInside, adjacency checks).
    - Room effects (PoisonRoom and TrapRoom correctly update health, poison status, and remaining moves).
    - Player poison logic and death detection.
- **Scenario-level tests:**
    - Player successfully reaches the exit under normal conditions.
    - Player dies due to traps or poison.
    - Monster catches the player according to the designed movement strategy.

Observed behavior is compared with the expected flows from Figures 3.2–3.4 and the state transitions from Figures 3.7–3.8, thereby validating that the code correctly realizes the object-oriented design.

CHAPTER 5 : CONCLUSION

5.1 Summary of the Work

The Cave Run project applied object-oriented analysis, design, and programming techniques to engineer a small but non-trivial game using C++.
Requirements were captured through use cases and conceptual models, refined into UML design diagrams, and then implemented as a set of cooperating classes that realize the specified behavior.

The system supports a grid-based cave with normal, poisonous, and trap rooms, a player with limited health and movement, and an autonomous monster that reacts to the player and exit positions.
Core OO concepts such as inheritance, polymorphism, and encapsulation were used to structure game entities and to separate game rules from the user interface.

5.2 Achievement of Objectives

The main objective of designing and implementing Cave Run using OOAD and OOP was achieved by producing a full set of UML diagrams and a corresponding C++ implementation. Analysis-level artifacts included clearly defined functional and non-functional requirements, actors, use cases, and conceptual classes, while design-level artifacts covered detailed class, sequence, communication, and state diagrams.

On the implementation side, the class hierarchies for characters and rooms, the game loop, and the map management were coded in C++ in a way that closely follows the design model.
The game logic runs through a structured turn system, and the user interface, whether console-based or graphical, interacts with the core model through well-defined operations.

5.3 Limitations of the System

The current implementation focuses on core gameplay and educational clarity rather than advanced game features or production-level polish.
Graphics, animation, and sound are simplified, particularly in a console-based version where the cave is rendered as text rather than a rich visual environment.

The monster's behavior follows a straightforward distance-based strategy and does not incorporate more complex pathfinding or adaptive AI techniques.
Additionally, there is no support for multiple levels, saving and loading game state, or multiplayer modes, which limits replayability compared to commercial games.

5.4 Recommendations and Future Enhancements

Several enhancements could be implemented with relatively small changes thanks to the modular design and use of polymorphism.
New room types such as healing rooms, bonus rooms, or teleport rooms could be added as subclasses of Room without altering the existing game loop, demonstrating the flexibility of the OO architecture.

The user interface could be improved by adopting a dedicated 2D graphics library, adding animations, sound effects, and clearer visual feedback for hazards and status changes.
More sophisticated AI for the monster, such as pathfinding algorithms or adaptive behavior, and additional game modes (multiple players, timed challenges) would further increase the game's depth and educational value.

5.5 Reflections on OOAD and OOP

This project illustrates how OOAD and OOP help to manage complexity even in small games by organizing behavior around coherent classes and clearly defined interactions.
Working from analysis through design to implementation showed the importance of consistent models: use cases guided the interaction diagrams, which in turn guided the C++ class structure and method interfaces.

The experience also highlighted practical challenges, such as choosing appropriate levels of abstraction and keeping diagrams and code synchronized, which are typical in real software engineering projects.
Overall, the Cave Run game demonstrates that disciplined use of OO techniques leads to a system that is easier to understand, extend, and maintain than an ad hoc procedural implementation.

REFERENCES

Booch, G., Maksimchuk, R. A., Engel, M. W., Young, B. J., Conallen, J., & Houston, K. A. (2007). *Object-oriented analysis and design with applications* (3rd ed.). Addison-Wesley. (Excerpt accessed from https://zjnu2017.github.io/OOAD/reading/Object.Oriented.Analysis.and.Design.with.Applications.3rd.Edition.by.Booch.pdf

GeeksforGeeks. (2023, November 23). *Use case diagram – Unified Modeling Language (UML)*. Retrieved from https://www.geeksforgeeks.org/system-design/use-case-diagram/

GeeksforGeeks. (2018, August 29). *UML class diagram*. Retrieved from https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-class-diagrams/

Nulab. (2025, March 23). *UML diagrams: A practical guide for software professionals*. Retrieved from https://nulab.com/learn/software-development/uml-diagrams-guide/

Oregon State University. (2023). *Unified Modeling Language class and sequence diagrams*. In *Software engineering textbook* (online open textbook). Retrieved from https://open.oregonstate.education/setextbook/chapter/uml-class/

Purdue University. (n.d.). *Object-oriented software engineering* (Lecture notes CS 307). Retrieved from https://www.cs.purdue.edu/homes/cs307/slides/lecture11_2up.pdf

Stack Overflow. (2014, September 29). *Use case diagram for board game – UML* [Question and answers]. Retrieved from https://stackoverflow.com/questions/26127105/use-case-diagram-for-board-game

Studocu. (2023, October 10). *Game development project report using C++* [Student report]. University of Mumbai. Retrieved from https://www.studocu.com/in/document/university-of-mumbai/computer-network/project/72189742

Habtamu Keno. (2020). *Object oriented system analysis and design (OOSAD) module* [PDF]. Retrieved from http://ndl.ethernet.edu.et/bitstream/123456789/90366/3/Revised%20OOSAD%20Module2020.pdf

Nehul Singh. (2021, May 17). *Object oriented programming concepts: Limitations and application trends*. Retrieved from https://www.scribd.com/document/837448880/Object-Oriented-Programming-Concepts-Limitations-and-Application-Trends

Mark Overmars. (2004). *Learning object-oriented design by creating games* (Technical report). Utrecht University. Retrieved from https://webdoc.sub.gwdg.de/ebook/serien/ah/UU-CS/2004-057.pdf